TP Python

BTS CIEL — 1re année

Lycée Denis Diderot | Année 2025–2026

NOM : _____

Prénom : _____

Groupe : _____

TP	Thème	/10
1	Variables et types de données	
2	Structures conditionnelles	
3	Boucles `for` et `while`	
4	Fonctions	
5	Listes	
6	Chaînes de caractères	
7	Dictionnaires	
8	Fichiers et gestion d'erreurs	
9	`numpy` et `matplotlib`	
10	Projet : calculatrice électronique	
	Total	**/100**

TP 1 — Variables et types de données /10 pts

Contexte : loi d'Ohm $U = R \times I$ $P = U \times I$

Déclarer des variables, utiliser les types `int`, `float`, `str`, effectuer des calculs et afficher des résultats formatés.

Exemple : déclaration et affichage

```python
R = 470            # résistance en ohms (int)
I = 0.02           # courant en ampères (float)
U = R * I
print(f"Tension U = {U} V")
print(type(R))     # affiche <class 'int'>
```

Q1. [2 pts] Déclarer `R = 470` et `I = 0.02`, calculer et afficher la tension `U`. Vérifier que $U = 9{,}4$ V.

Q2. [2 pts] Utiliser `type()` pour afficher le type de `R`, `I` et d'une variable `nom = "Résistance"`.

Q3. [2 pts] Demander `R` et `I` à l'utilisateur avec `input()`, convertir en `float()`, recalculer et afficher `U`.

Q4. [2 pts] Calculer $P = U \times I$, afficher avec `round(P, 4)` et l'unité watts.

Q5. [2 pts] Afficher un résumé avec f-string : `R = ... Ω | I = ... A | U = ... V | P = ... W`

Grille d'évaluation – TP 1

Q	Critères observables	/pts	Note	Observations
1	• R=470 et I=0.02 déclarés • U = R * I calculé • Affichage correct : U = 9.4 V	2		
2	• type() appelé sur R, I et nom • int, float, str identifiés	2		
3	• input() utilisé pour R et I • Conversion float() appliquée • U recalculé et affiché	2		
4	• P = U * I calculé • round(P, 4) utilisé • Unité « watts » présente	2		
5	• f-string utilisé • R, I, U, P avec unités (Ω, A, V, W)	2		
		Total TP 1	**/10**	

TP 2 — Structures conditionnelles /10 pts

Contexte : seuils de tension dans un circuit électronique

Utiliser `if`, `elif`, `else` et les opérateurs logiques `and`, `or` pour prendre des décisions.

Exemple : structure conditionnelle

```python
U = float(input("Tension U (V) : "))
if U > 5.5:
    print("Tension élevée")
elif U >= 4.5:
    print("Tension nominale")
else:
    print("Tension faible")
```

Q1. [2 pts] Une LED est passante si $U > 2{,}0$ V. Écrire un programme qui affiche `"LED allumée"` ou `"LED éteinte"` selon la valeur saisie.

Q2. [2 pts] Classifier la tension de sortie d'un régulateur en 3 zones : **faible** ($U < 4{,}5$ V), **nominale** ($4{,}5 \leq U \leq 5{,}5$ V), **élevée** ($U > 5{,}5$ V). Utiliser `elif`.

Q3. [2 pts] Un système se déclenche si $U > 12$ V **ET** $I > 2$ A. Utiliser and. Tester 3 combinaisons.

Q4. [2 pts] Afficher la table de vérité d'une porte AND pour $A, B \in \{0, 1\}$ avec deux boucles `for` imbriquées.

Q5. [2 pts] Un thermostat coupe le chauffage si $T > 22$°C, l'allume si $T < 18$°C, affiche `"Maintien"` sinon.

Grille d'évaluation – TP 2

Q	Critères observables	/pts	Note	Observations
1	• `if` / `else` présents • Seuil 2,0 V correct • Affichage `"LED allumée"` / `"LED éteinte"`	2		
2	• `elif` utilisé (3 branches) • Seuils 4,5 V et 5,5 V corrects • Libellés « faible / nominale / élevée »	2		
3	• and utilisé • Seuils U > 12 V et I > 2 A • 3 combinaisons testées	2		
4	• Deux `for` imbriqués (A et B dans {0,1}) • Condition AND correcte • 4 lignes de table de vérité	2		
5	• Seuils 18 °C et 22 °C corrects • `"Maintien"` entre les deux bornes • `input()` utilisé	2		
	Total TP 2		**/10**	

TP 3 — Boucles `for` et `while` /10 pts

Contexte : mesures répétées et simulations

Maîtriser `for` (nombre d'itérations connu) et `while` (condition de sortie). Utiliser `range()`, accumuler des valeurs.

Exemple : exemples de boucles

```python
for i in range(1, 10, 2):    # 1, 3, 5, 7, 9
    print(i)

U = 100
compteur = 0
while U >= 1:
    U /= 2
    compteur += 1
```

Q1. [2 pts] Afficher les 12 premières valeurs de la série E12 en multipliant $R_0 = 10\ \Omega$ par $\times 1{,}2115$ à chaque tour. Arrondir à l'entier.

Q2. [2 pts] Avec une boucle `while`, diviser $U = 100$ V par 2 jusqu'à $U < 1$ V. Afficher le nombre d'itérations.

Q3. [2 pts] Calculer $\sum_{k=1}^{n} k$ avec une boucle `for`. Vérifier avec la formule $n(n+1)/2$.

Q4. [2 pts] Programme de devinette : l'ordinateur choisit un nombre entre 1 et 100 (`random.randint`), l'utilisateur propose jusqu'à trouver. Afficher le nombre de tentatives.

Q5. [2 pts] Afficher une table de multiplication de 1 à 5 avec deux boucles imbriquées. Aligner les colonnes avec `:>4d`.

Grille d'évaluation – TP 3

Q	Critères observables	/pts	Note	Observations
1	• Boucle `for` sur 12 itérations • Facteur $\times 1{,}2115$ appliqué • Arrondi à l'entier et affiché	2		
2	• `while U >= 1` correct • Division par 2 à chaque tour • Nombre d'itérations affiché	2		
3	• Accumulation dans un `for` • Somme correcte, vérif. $n(n+1)/2$ affichée	2		
4	• `random.randint(1, 100)` utilisé • Boucle `while` jusqu'à la bonne réponse • Nombre de tentatives affiché	2		
5	• Deux `for` imbriqués • Format `:>4d` pour l'alignement • Table complète et lisible	2		
		Total TP 3	**/10**	

TP 4 — Fonctions /10 pts

Contexte : bibliothèque de calculs électroniques réutilisables

Définir des fonctions avec `def`, passer des paramètres, utiliser `return`, les valeurs par défaut et retourner plusieurs résultats.

Exemple : définition et appel

```python
def loi_ohm(R, I):
    """Retourne U = R * I"""
    return R * I

tension = loi_ohm(470, 0.02)    # appel
print(f"U = {tension} V")
```

Q1. [2 pts] Écrire `loi_ohm(R, I)` qui retourne $U = R \times I$. Tester avec $R = 1000\ \Omega$ et $I = 0{,}015$ A.

Q2. [2 pts] Écrire `bilan_puissance(R, I)` qui retourne un tuple (U, P). Décompacter le résultat dans deux variables.

Q3. [2 pts] Écrire `resistance_serie(*args)` qui accepte un nombre quelconque de résistances et retourne leur somme. Tester avec 3 puis 5 valeurs.

Q4. [2 pts] Écrire `resistance_parallele(R1, R2=100)` avec valeur par défaut. Tester sans préciser `R2`.

Q5. [2 pts] Écrire `convertir_tension(U, unite="V")` qui convertit en mV, V ou kV. Tester les 3 cas.

Grille d'évaluation – TP 4

Q	Critères observables	/pts	Note	Observations
1	• `def loi_ohm(R, I):` définie • `return U` présent • Appel correct, résultat 15 V	2		
2	• Tuple `(U, P)` retourné • Décompactage `u, p = ...` correct	2		
3	• `*args` dans la signature • Somme correcte • Testé avec 3 et 5 résistances	2		
4	• Valeur par défaut `R2=100` • Formule $R_p = R_1R_2/(R_1 + R_2)$ • Appel sans `R2` fonctionnel	2		
5	• Paramètre `unite` présent • 3 conversions correctes • Valeur par défaut `"V"` opérationnelle	2		
		Total TP 4	**/10**	

TP 5 — Listes /10 pts

Contexte : traitement d'une série de mesures de tension

Créer, indexer, modifier des listes ; utiliser `append`, `remove`, `sort` ; parcourir avec `for` ; utiliser `min`, `max`, `sum`, `len`.

Exemple : manipulation de listes

```python
mesures = [4.98, 5.02, 4.95, 5.10, 4.88]
print(mesures[0])       # 4.98
print(mesures[-1])      # 4.88
mesures.append(5.05)
print(len(mesures))     # 6
```

Q1. [2 pts] Créer la liste `[4.98, 5.02, 4.95, 5.10, 4.88, 5.01, 4.97, 5.08]`. Afficher le 1er élément, le dernier et l'élément d'indice 4.

Q2. [2 pts] Ajouter `5.15` avec `append()`, supprimer `4.88` avec `remove()`. Afficher la liste et sa longueur.

Q3. [2 pts] Calculer et afficher la moyenne, la valeur minimale et maximale avec `sum()`, `min()`, `max()`, `len()`.

Q4. [2 pts] Créer `hors_limites` contenant les mesures en dehors de [4,90 V;5,10 V] à l'aide d'une boucle `for`.

Q5. [2 pts] Trier la liste avec `sort()` et extraire le premier tiers avec le slicing.

Grille d'évaluation – TP 5

Q	Critères observables	/pts	Note	Observations
1	• Liste de 8 éléments créée • Accès `[0]`, `[-1]`, `[4]` corrects	2		
2	• `append(5.15)` utilisé • `remove(4.88)` utilisé • `len()` correct	2		
3	• Moyenne avec `sum()/len()` • `min()` et `max()` corrects	2		
4	• Boucle `for` + condition hors [4,90 ; 5,10] V • `append()` dans `hors_limites`	2		
5	• `sort()` appliqué, liste triée affichée • Slicing premier tiers correct	2		
		Total TP 5	**/10**	

TP 6 — Chaînes de caractères /10 pts

Contexte : analyse de trames de communication série ASCII

Concaténation, indexation, méthodes `upper`/`lower`, `split`, `strip`, `startswith`, `find`, formatage avec f-strings.

Exemple : méthodes principales

```python
trame = "START:12.5:STOP"
print(trame.split(":"))        # ['START', '12.5', 'STOP']
print(trame.startswith("ST")) # True
print(trame[6:9])              # 12.
```

Q1. [2 pts] Déclarer `trame = "START:12.5:STOP"`. Afficher sa longueur, le 1er caractère, le dernier et les caractères d'indices 6 à 8.

Q2. [2 pts] Décomposer la trame avec `split(":")`, stocker dans `parties` et afficher séparément début, valeur et fin.

Q3. [2 pts] Écrire `valider_trame(t)` qui retourne `True` si la trame commence par `"START"` et finit par `"STOP"`. Tester 3 trames.

Q4. [2 pts] Construire avec f-string : `"[INFO] Capteur 3 - Valeur : 12.50 V - Statut : OK"` à partir de variables `num=3`, `val=12.5`, `statut="OK"`.

Q5. [2 pts] Écrire `compter_char(chaine, car)` qui compte les occurrences de `car` sans `.count()`. Tester en comptant les `":"` dans la trame.

Grille d'évaluation – TP 6

Q	Critères observables	/pts	Note	Observations
1	• `len(trame)` correct (15) • `trame[0]`, `trame[-1]` corrects • Slicing `trame[6:9]` correct	2		
2	• `split(":")` utilisé • 3 parties stockées et affichées	2		
3	• `startswith` et `endswith` utilisés • Retourne `True`/`False` • 3 trames testées	2		
4	• f-string utilisé • Variables `num`, `val`, `statut` insérées • Format exact	2		
5	• Boucle `for` sans `.count()` • Résultat correct	2		
		Total TP 6	**/10**	

TP 7 — Dictionnaires /10 pts

Contexte : catalogue de composants électroniques

Créer, lire, modifier des dictionnaires ; utiliser `keys()`, `values()`, `items()` ; travailler avec des dictionnaires imbriqués.

Exemple : dictionnaire simple

```python
composant = {"référence": "R470", "valeur": 470, "tolérance": 5}
print(composant["valeur"])    # 470
composant["tolérance"] = 1    # modification
```

Q1. [2 pts] Créer un dictionnaire `resistances` avec 4 entrées (`"R1"` à `"R4"`). Afficher la valeur de `"R3"` et le nombre total d'entrées.

Q2. [2 pts] Ajouter `"R5": 2200`, supprimer `"R2"` avec `del`. Vérifier avec `in` que `"R2"` n'est plus présent.

Q3. [2 pts] Parcourir avec `.items()` et afficher chaque résistance au format : `Référence : R1 | Valeur : 470` Ω.

Q4. [2 pts] Créer un dictionnaire imbriqué où chaque composant a les attributs `valeur`, `tolérance` et `stock`. Afficher le stock du 2e composant.

Q5. [2 pts] Écrire `chercher_stock(catalogue, seuil)` qui retourne les références dont le stock est inférieur au seuil. Tester avec `seuil=10`.

Grille d'évaluation – TP 7

Q	Critères observables	/pts	Note	Observations
1	• Dict avec 4 résistances créé • Accès `["R3"]` correct, `len()` = 4	2		
2	• Ajout `"R5"` opérationnel • `del` exécuté • `"R2" in dict` → `False`	2		
3	• `.items()` dans la boucle • Format d'affichage correct	2		
4	• Dict imbriqué avec 3 attributs • Accès stock du 2e composant correct	2		
5	• Fonction définie, filtre correct • Test `seuil=10` fonctionnel	2		
	Total TP 7		**/10**	

TP 8 — Fichiers et gestion d'erreurs /10 pts

Contexte : datalogger – enregistrement de mesures

Ouvrir, lire, écrire des fichiers avec `open()` et `with` ; utiliser le module `csv` ; gérer les erreurs avec `try / except`.

Exemple : lecture et écriture

```python
with open("mesures.txt", "w", encoding="utf-8") as f:
    f.write("5.02\n")

with open("mesures.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())
```

Q1. [2 pts] Créer `log.txt` et y écrire 5 tensions générées avec `random.uniform(4.9, 5.1)`.

Q2. [2 pts] Lire `log.txt` ligne par ligne, convertir en `float` et calculer la moyenne. Afficher le résultat.

Q3. [2 pts] Utiliser `csv.writer` pour créer `mesures.csv` avec 3 colonnes (index, temps_s, tension_V) et 10 lignes (`temps = index * 0.1`).

Q4. [2 pts] Lire `mesures.csv` avec `csv.DictReader` et afficher uniquement les lignes où la tension est hors de $[4{,}90\ V ; 5{,}10\ V]$.

Q5. [2 pts] Entourer la lecture d'un `try / except FileNotFoundError` avec un message clair. Tester avec un nom de fichier inexistant.

Grille d'évaluation – TP 8

Q	Critères observables	/pts	Note	Observations
1	• `with open(..., "w")` utilisé • `random.uniform(4.9, 5.1)` • 5 lignes dans `log.txt`	2		
2	• Lecture ligne par ligne • `float()` appliqué • Moyenne calculée et affichée	2		
3	• `csv.writer` utilisé • 3 colonnes, 10 lignes, `temps = index * 0.1`	2		
4	• `csv.DictReader` utilisé • Filtre hors [4,90 ; 5,10] V correct	2		
5	• `try / except FileNotFoundError` présent • Message clair, testé avec fichier inexistant	2		
		Total TP 8	**/10**	

TP 9 — Modules `numpy` et `matplotlib` /10 pts

Contexte : visualisation d'un signal sinusoïdal

Utiliser `numpy` pour les calculs vectoriels et `matplotlib.pyplot` pour tracer des courbes. Générer $u(t) = A\sin(2\pi f t)$ et visualiser.

Exemple : signal sinusoïdal de base

```python
import numpy as np
import matplotlib.pyplot as plt

f, A = 50, 5
t = np.linspace(0, 0.04, 1000)
u = A * np.sin(2 * np.pi * f * t)
plt.plot(t, u)
plt.show()
```

Q1. [2 pts] Créer un tableau de 1000 points sur [0 ; 0,04 s] avec `np.linspace`. Calculer $u = 5\sin(2\pi \cdot 50 \cdot t)$. Afficher les 5 premières valeurs.

Q2. [2 pts] Tracer $u(t)$ avec `plt.plot()`. Ajouter un titre, des labels d'axes et une grille.

Q3. [2 pts] Superposer $u_1 = 5\sin(2\pi \cdot 50 \cdot t)$ et $u_3 = 1{,}5\sin(2\pi \cdot 150 \cdot t)$ en couleurs différentes.

Q4. [2 pts] Tracer le signal composite $u = u_1 + u_3$. Ajouter une légende avec `plt.legend()` pour les 3 courbes.

Q5. [2 pts] Calculer `np.max(u)` et $U_{\text{eff}} = \sqrt{\text{mean}(u^2)}$. Vérifier que $U_{\text{eff}} \approx A/\sqrt{2}$.

Grille d'évaluation – TP 9

Q	Critères observables	/pts	Note	Observations
1	• `np.linspace(0, 0.04, 1000)` correct • $u = A\sin(2\pi f t)$ vectoriel • `print(u[:5])` affiché	2		
2	• `plt.plot(t, u)` présent • Titre + labels + `plt.grid()`	2		
3	• u_1 et u_3 calculés • 2 courbes superposées, couleurs différentes	2		
4	• $u = u_1 + u_3$ par addition de tableaux • `plt.legend()` avec 3 courbes nommées	2		
5	• `np.max(u)` affiché • $U_{\text{eff}} = \sqrt{\text{mean}(u^2)}$ calculée • Vérification $\approx A/\sqrt{2}$ affichée	2		
		Total TP 9	**/10**	

TP 10 — Projet : calculatrice électronique interactive /10 pts

Bilan des compétences – application complète

Concevoir une application complète mobilisant toutes les compétences : fonctions, boucles, conditions, fichiers, formatage. Le code doit être structuré et commenté.

Q1. [2 pts] Écrire `menu()` qui affiche les 4 options + quitter (0) et retourne le choix. Gérer les saisies non entières avec `try/except ValueError`.

Q2. [2 pts] Écrire `module_loi_ohm()` : demander 2 grandeurs parmi U, R, I, calculer la 3e et afficher le résultat avec unité.

Q3. [2 pts] Écrire `module_diviseur()` : calculer $U_s = U_e \times \dfrac{R_2}{R_1 + R_2}$. Saisir U_e, R_1, R_2 et afficher U_s.

Q4. [2 pts] Écrire `module_led()` : calculer $R = (U_{\text{alim}} - U_{\text{LED}})/I_{\text{LED}}$ avec valeurs par défaut $U_{\text{LED}} = 2{,}0$ V et $I_{\text{LED}} = 0{,}02$ A.

Q5. [2 pts] Assembler dans une boucle `while True`. Enregistrer chaque calcul dans `historique.txt` avec `datetime.now()`.

Grille d'évaluation – TP 10

Q	Critères observables	/pts	Note	Observations
1	• Menu affiché avec 4 options + quitter • `while True` présent • `try/except ValueError` opérationnel	2		
2	• Fonction `module_loi_ohm()` définie • 2 grandeurs saisies, 3e calculée • Résultat avec unité affiché	2		
3	• Formule $U_s = U_e R_2/(R_1 + R_2)$ correcte • Saisies et affichage opérationnels	2		
4	• Formule $R = (U_{\text{alim}} - U_{\text{LED}})/I_{\text{LED}}$ correcte • Valeurs par défaut 2,0 V et 0,02 A	2		
5	• 4 modules appelés selon le choix • `historique.txt` en mode ajout `"a"` • `datetime.now()` enregistré	2		
	Total TP 10		**/10**	